

Assignment 3 | Bridge Pattern

Course: ShP-2216 Software Design Patterns, Astana IT University

Student: Baktybay Nazar **Group:** SE-2530 **Topic:** A (Drawing: Shape / Renderer)

Repository: <https://github.com/Nazar887667/SDP3assignment.git>

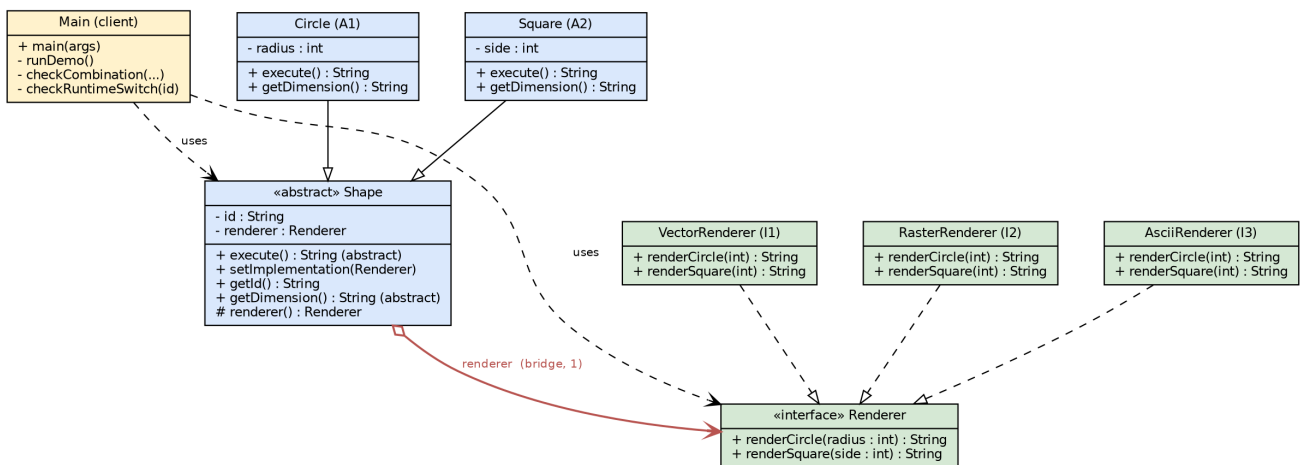
Base commit (I1/I2 version): 914ec7a5e65d97e3fe7734f4803d2fa2a94ee32a

I3 extension commit: 775fb4e3abfc556dbbce453c3005970499ca1dcc (source identical in the final documentation commit, whose hash is given in the Moodle text)

1. Two independent dimensions and why Bridge fits

The program varies along two axes. The **abstraction** axis is the kind of shape: Circle (radius 2) and Square (side 3). The **implementation** axis is how a shape is rendered: VectorRenderer, RasterRenderer and, later, AsciiRenderer. Without Bridge, every pair would need its own class ($2 \times 3 = 6$ now, and a new shape or renderer multiplies the count). Bridge keeps two small hierarchies and links them by composition: Shape holds a Renderer reference, supplied in the constructor and replaceable through `setImplementation(...)`. Either side can grow without touching the other.

2. UML class diagram



The red link is the bridge: the interface-typed field `renderer` in `Shape`. `Main` depends only on the public types.

3. Clean Code excerpts

Separation of responsibilities (Shape.java)

```
private final String id;
private Renderer renderer; // the bridge
public abstract String execute();
```

Benefit: Shape knows what is drawn and which renderer is plugged in; the renderers know only how to produce output. Neither side changes when the other does.

Meaningful names (Renderer.java)

```
String renderCircle(int radius);
String renderSquare(int side);
```

Benefit: Method and parameter names state the operation and the unit of data, so the code reads without comments.

Small focused methods (Circle.java)

```
public String execute() {
    return renderer().renderCircle(radius);
}
```

Benefit: One line, one job: delegate through the bridge. Behaviour is easy to trace and to test.

No duplicated workflow (Main.java)

```
private static void checkCombination(String id,
    Function<Renderer, Shape> factory, Renderer renderer, String expected)
```

Benefit: T1-T4, T6 and T7 share one helper that builds, runs, compares and prints. Adding T6/T7 needed only two calls, not new logic.

Encapsulated state (Circle.java / Shape.java)

```
private final int radius; // Circle
private final String id; // Shape
protected Renderer renderer() // subclass-only access
```

Benefit: Fields are private; the renderer is reachable only by subclasses and can change only through `setImplementation(...)`, which rejects null. Domain data is final, so a runtime switch cannot corrupt it.

4. Adding I3, trade-off and Adapter comparison

Extension. After committing the working I1/I2 version (base commit above), I added only `bridge/AsciiRenderer.java` (implements `Renderer`) and three lines in `Main` for T6 and T7, plus `sources.txt` and documentation. `Shape`, `Circle`, `Square`, `Renderer`, `VectorRenderer` and `RasterRenderer` are unchanged; `extension.diff` shows 2 files changed within `src/`.

Trade-off. Bridge adds indirection and an extra interface. If a new shape needs a rendering operation the interface lacks (for example a triangle), the `Renderer` contract must grow and every renderer must be updated.

Bridge vs Adapter. Bridge is a *design-time* structure that separates an abstraction from its implementations so both evolve independently; here `Shape` and `Renderer` were designed together. Adapter is applied *after* the fact to make an existing, incompatible class fit an expected interface (for example wrapping a third-party drawing library behind `Renderer`). Bridge plans for variation; Adapter repairs a mismatch.

Evidence. `demo-output.txt` records the real run: T1-T7 all PASS, SUMMARY: 7/7 PASS.

5. Sources

[1] Gamma, Helm, Johnson, Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994 (Bridge, Adapter). [2] Martin, R. C. *Clean Code*, Prentice Hall, 2008. [3] Lecture 4, Bridge Pattern, ShP-2216, Astana IT University, 2026-2027. [4] Course syllabus 2026-2027 and Bridge reading in Moodle.